

CS722L5C: DEEP LEARNING

10 Laboratory Programs

1. Design and implement a deep feedforward neural network to learn the XOR function using gradient-based learning and hidden units.

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.optimizers import Adam
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 1, 1, 0])
model = Sequential()
model.add(Input(shape=(2,)))
model.add(Dense(2, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(
    loss='binary_crossentropy',
    optimizer=Adam(learning_rate=0.01),
    metrics=['accuracy']
)
model.fit(X, y, epochs=1000, verbose=0)
loss, accuracy = model.evaluate(X, y)
print(f"Accuracy: {accuracy*100:.2f}%")
predictions = model.predict(X)
predictions = (predictions > 0.5).astype(int)
print("\nPredictions:")
for i, prediction in enumerate(predictions):
    print(f"Input: {X[i]} - Predicted Output: {prediction[0]} - True Output: {y[i]}")
```

2. Implement regularization techniques in deep learning models using parameter norm penalties, dataset augmentation, and noise robustness for improved generalization.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers
import numpy as np
(x_train, y_train), _ = keras.datasets.mnist.load_data()
x_train = x_train.astype('float32') / 255
x_train = np.expand_dims(x_train, -1)
y_train = tf.keras.utils.to_categorical(y_train, 10)
noise = 0.05 * np.random.normal(size=x_train.shape)
x_train = np.clip(x_train + noise, 0., 1.)
datagen = keras.preprocessing.image.ImageDataGenerator(
    rotation_range=10,
    width_shift_range=0.1,
    height_shift_range=0.1,
    validation_split=0.2
)
datagen.fit(x_train)
model = keras.Sequential([
    layers.Flatten(input_shape=(28, 28, 1)),
    layers.Dense(
        256,
        activation='relu',
        kernel_regularizer=regularizers.l1_l2(l1=1e-5, l2=1e-4)
    ),
    layers.Dropout(0.5),
    layers.Dense(
        128,
        activation='relu',
        kernel_regularizer=regularizers.l2(1e-4)
    ),
    layers.Dropout(0.3),
    layers.Dense(10, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
early_stop = keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)
history = model.fit(
    datagen.flow(
        x_train,
        y_train,
        batch_size=128,
        subset='training'
    ),
    validation_data=datagen.flow(
        x_train,
        y_train,
        batch_size=128,
        subset='validation'
    ),
    epochs=50,
    callbacks=[early_stop]
)
```

3. Implement and compare different optimization algorithms (SGD, Momentum, Adam) for training a simple neural network on a toy dataset.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, optimizers, losses
import numpy as np
import matplotlib.pyplot as plt
tf.random.set_seed(42)
np.random.seed(42)
X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=np.float32)
y = np.array([[0],[1],[1],[0]], dtype=np.float32)
def create_model():
    model = keras.Sequential([
        layers.Dense(4, input_dim=2, activation='tanh'),
        layers.Dense(1, activation='sigmoid')
    ])
    return model
def train_model(optimizer_name, X, y, epochs=5000, lr=0.1):
    model = create_model()
    if optimizer_name == 'SGD':
        optimizer = optimizers.SGD(learning_rate=lr)
    elif optimizer_name == 'Momentum':
        optimizer = optimizers.SGD(learning_rate=lr, momentum=0.9)
    elif optimizer_name == 'Adam':
        optimizer = optimizers.Adam(learning_rate=lr)
    else:
        raise ValueError(f"Unknown optimizer: {optimizer_name}")
    model.compile(
        optimizer=optimizer,
        loss=losses.BinaryCrossentropy()
    )
    history = model.fit(X, y, epochs=epochs, verbose=0)
    return history.history['loss']
losses_sgd = train_model('SGD', X, y)
losses_momentum = train_model('Momentum', X, y)
losses_adam = train_model('Adam', X, y)
plt.figure(figsize=(8, 5))
plt.plot(losses_sgd, label='SGD')
plt.plot(losses_momentum, label='Momentum')
plt.plot(losses_adam, label='Adam')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('Optimization Algorithms Comparison (XOR Problem)')
plt.show()
print(f"Final Loss (SGD): {losses_sgd[-1]:.6f}")
print(f"Final Loss (Momentum): {losses_momentum[-1]:.6f}")
print(f"Final Loss (Adam): {losses_adam[-1]:.6f}")
```

4. Build a CNN on MNIST dataset to demonstrate convolution, pooling, and classification.

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt

(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.reshape(
    (x_train.shape[0], 28, 28, 1)
).astype('float32') / 255
x_test = x_test.reshape(
    (x_test.shape[0], 28, 28, 1)
).astype('float32') / 255
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
model = models.Sequential([
    layers.Conv2D(
        32,
        (3, 3),
        activation='relu',
        input_shape=(28, 28, 1)
    ),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(
        64,
        (3, 3),
        activation='relu'
    ),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test accuracy: {test_acc:.4f}")
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title("Training and Validation Accuracy")
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()
```

5. Implement an LSTM-based RNN for text classification to demonstrate sequence modeling, unfolding computational graphs, and handling long-term dependencies.

```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
import matplotlib.pyplot as plt

vocab_size = 10000
max_sequence_length = 200
(x_train, y_train), (x_test, y_test) = imdb.load_data(
    num_words=vocab_size
)
x_train = pad_sequences(
    x_train,
    maxlen=max_sequence_length,
    padding='post'
)
x_test = pad_sequences(
    x_test,
    maxlen=max_sequence_length,
    padding='post'
)
model = Sequential()
model.add(
    Embedding(
        input_dim=vocab_size,
        output_dim=64,
        input_length=max_sequence_length
    )
)
model.add(
    LSTM(
        units=64,
        return_sequences=False
    )
)
model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid'))
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
print("\nTraining...")
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)
print("\nEvaluating on test set...")
loss, accuracy = model.evaluate(
    x_test,
    y_test
)
print(f"\nTest Accuracy: {accuracy:.4f}")
def plot_graphs(history, string):
    plt.plot(history.history[string])
    plt.plot(history.history['val_' + string])
    plt.xlabel("Epochs")
    plt.ylabel(string.capitalize())
```

```
plt.title(f'{string.capitalize()} Over Epochs')
plt.legend([string, 'val_' + string])
plt.grid(True)
plt.show()
plot_graphs(history, "accuracy")
plot_graphs(history, "loss")
```

6. Construct a Bidirectional RNN and compare its performance with a standard RNN on sequence prediction tasks.

```
import tensorflow as tf
import numpy as np
seq_length = 10
num_samples = 1000
X = []
y = []
for _ in range(num_samples):
    start = np.random.randint(0, 100)
    seq = np.arange(start, start + seq_length)
    X.append(seq[:-1])
    y.append(seq[1:])
X = np.array(X, dtype=np.float32)[..., np.newaxis]
y = np.array(y, dtype=np.float32)[..., np.newaxis]
def build_simple_rnn(input_size=1, hidden_size=32, output_size=1):
    model = tf.keras.Sequential([
        tf.keras.layers.SimpleRNN(
            hidden_size,
            return_sequences=True,
            input_shape=(seq_length - 1, input_size)
        ),
        tf.keras.layers.Dense(output_size)
    ])
    model.compile(
        optimizer=tf.keras.optimizers.Adam(0.01),
        loss='mse'
    )
    return model
def build_birnn(input_size=1, hidden_size=32, output_size=1):
    model = tf.keras.Sequential([
        tf.keras.layers.Bidirectional(
            tf.keras.layers.SimpleRNN(
                hidden_size,
                return_sequences=True
            ),
            input_shape=(seq_length - 1, input_size)
        ),
        tf.keras.layers.Dense(output_size)
    ])
    model.compile(
        optimizer=tf.keras.optimizers.Adam(0.01),
        loss='mse'
    )
    return model
def train_model(model, X, y, epochs=50):
    history = model.fit(
        X,
        y,
        epochs=epochs,
        verbose=0
    )
    return history.history['loss'][-1]
rnn_loss = train_model(build_simple_rnn(), X, y)
birnn_loss = train_model(build_birnn(), X, y)
print(f"Final Loss (RNN): {rnn_loss:.4f}")
print(f"Final Loss (BiRNN): {birnn_loss:.4f}")
```

7. Implement an encoder-decoder architecture with LSTMs for sequence-to-sequence learning.

```
import numpy as np
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, LSTM, Dense
num_samples = 1000
timesteps = 5
input_dim = 1
latent_dim = 32
X = np.random.rand(
    num_samples,
    timesteps,
    input_dim
)
Y = np.flip(X, axis=1)
encoder_inputs = Input(
    shape=(timesteps, input_dim)
)
encoder_outputs, state_h, state_c = LSTM(
    latent_dim,
    return_state=True
)(encoder_inputs)
encoder_states = [state_h, state_c]
decoder_inputs = Input(
    shape=(timesteps, input_dim)
)
decoder_lstm = LSTM(
    latent_dim,
    return_sequences=True,
    return_state=True
)
decoder_outputs, _, _ = decoder_lstm(
    decoder_inputs,
    initial_state=encoder_states
)
decoder_dense = Dense(input_dim)
decoder_outputs = decoder_dense(
    decoder_outputs
)
model = Model(
    [encoder_inputs, decoder_inputs],
    decoder_outputs
)
model.compile(
    optimizer='adam',
    loss='mse'
)
model.fit(
    [X, Y],
    Y,
    epochs=10,
    batch_size=32,
    verbose=1
)
pred = model.predict(
    [X[:1], Y[:1]]
)
print("Input:\n", X[0].squeeze())
print("Predicted reversed:\n", pred[0].squeeze())
print("True reversed:\n", Y[0].squeeze())
```

8. Implement a Restricted Boltzmann Machine (RBM) for learning binary data Representations.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
(x_train, _) = tf.keras.datasets.mnist.load_data()
x_train = x_train.astype('float32') / 255.0
x_train = (x_train > 0.5).astype('float32')
x_train = x_train.reshape(-1, 784)
batch_size = 64
train_dataset = tf.data.Dataset.from_tensor_slices(
    x_train
).shuffle(10000).batch(batch_size)
class RBM(tf.keras.Model):
    def __init__(self, n_visible, n_hidden):
        super(RBM, self).__init__()
        self.n_visible = n_visible
        self.n_hidden = n_hidden
        initializer = tf.initializers.RandomNormal(
            mean=0.0,
            stddev=0.01
        )
        self.W = tf.Variable(
            initializer([n_visible, n_hidden]),
            name='weights'
        )
        self.h_bias = tf.Variable(
            tf.zeros([n_hidden]),
            name='hidden_bias'
        )
        self.v_bias = tf.Variable(
            tf.zeros([n_visible]),
            name='visible_bias'
        )
    def sample_prob(self, probs):
        return tf.nn.relu(
            tf.sign(
                probs - tf.random.uniform(tf.shape(probs))
            )
        )
    def sample_h(self, v):
        prob_h = tf.nn.sigmoid(
            tf.matmul(v, self.W) + self.h_bias
        )
        return prob_h, self.sample_prob(prob_h)
    def sample_v(self, h):
        prob_v = tf.nn.sigmoid(
            tf.matmul(
                h,
                tf.transpose(self.W)
            ) + self.v_bias
        )
        return prob_v, self.sample_prob(prob_v)
    def contrastive_divergence(self, v, lr=0.01):
        prob_h, h0 = self.sample_h(v)
        prob_v, v1 = self.sample_v(h0)
        prob_h1, _ = self.sample_h(v1)
        positive_grad = tf.matmul(
            tf.transpose(v),
            prob_h
        )
        negative_grad = tf.matmul(
            tf.transpose(v1),
```

```

        prob_h1
    )
    batch_size = tf.cast(
        tf.shape(v)[0],
        tf.float32
    )
    self.W.assign_add(
        lr * (positive_grad - negative_grad) / batch_size
    )
    self.v_bias.assign_add(
        lr * tf.reduce_mean(
            v - v1,
            axis=0
        )
    )
    self.h_bias.assign_add(
        lr * tf.reduce_mean(
            prob_h - prob_h1,
            axis=0
        )
    )
    loss = tf.reduce_mean(
        tf.square(v - v1)
    )
    return loss
n_visible = 784
n_hidden = 128
rbm = RBM(
    n_visible,
    n_hidden
)
n_epochs = 5
lr = 0.05
losses = []
for epoch in range(n_epochs):
    epoch_loss = 0
    for batch in train_dataset:
        loss = rbm.contrastive_divergence(
            batch,
            lr
        )
        epoch_loss += loss.numpy()
    avg_loss = epoch_loss / len(
        list(train_dataset)
    )
    losses.append(avg_loss)
    print(
        f"Epoch {epoch+1}/{n_epochs}, "
        f"Reconstruction Loss: {avg_loss:.4f}"
    )
plt.figure(figsize=(7, 5))
plt.plot(
    range(1, n_epochs + 1),
    losses,
    marker='o'
)
plt.title("RBM Reconstruction Loss over Epochs")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.grid(True)
plt.show()

```

9. Develop a Denoising Autoencoder to reconstruct clean images from noisy input data.

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
x_train = x_train / 255.0
x_test = x_test / 255.0
x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)
x_train_noisy = np.clip(
    x_train + 0.3 * np.random.normal(
        0,
        1,
        x_train.shape
    ),
    0,
    1
)
x_test_noisy = np.clip(
    x_test + 0.3 * np.random.normal(
        0,
        1,
        x_test.shape
    ),
    0,
    1
)
plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(2, 10, i + 1)
    plt.imshow(
        x_test[i].squeeze(),
        cmap='gray'
    )
    plt.axis('off')
    plt.subplot(2, 10, i + 11)
    plt.imshow(
        x_test_noisy[i].squeeze(),
        cmap='gray'
    )
    plt.axis('off')
plt.show()
autoencoder = models.Sequential([
    layers.Conv2D(
        32,
        (3, 3),
        activation='relu',
        padding='same',
        input_shape=(28, 28, 1)
    ),
    layers.MaxPooling2D(
        (2, 2),
        padding='same'
    ),
    layers.Conv2D(
        64,
        (3, 3),
        activation='relu',
        padding='same'
    ),
    layers.MaxPooling2D(
        (2, 2),
        padding='same'
    ),
```

```

),
layers.Conv2D(
    64,
    (3, 3),
    activation='relu',
    padding='same'
),
layers.UpSampling2D((2, 2)),
layers.Conv2D(
    32,
    (3, 3),
    activation='relu',
    padding='same'

),
layers.UpSampling2D((2, 2)),
layers.Conv2D(
    1,
    (3, 3),
    activation='sigmoid',
    padding='same'
)
])
autoencoder.compile(
    optimizer='adam',
    loss='binary_crossentropy'
)
autoencoder.fit(
    x_train_noisy,
    x_train,
    epochs=5,
    batch_size=256,
    validation_data=(x_test_noisy, x_test)
)
decoded = autoencoder.predict(
    x_test_noisy
)
plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(3, 10, i + 1)
    plt.imshow(
        x_test[i].squeeze(),
        cmap='gray'
    )
    plt.axis('off')
    plt.subplot(3, 10, i + 11)
    plt.imshow(
        x_test_noisy[i].squeeze(),
        cmap='gray'
    )
    plt.axis('off')
    plt.subplot(3, 10, i + 21)
    plt.imshow(
        decoded[i].squeeze(),
        cmap='gray'
    )
    plt.axis('off')
plt.show()

```

10. Implement a simple Generative Adversarial Network (GAN) on the MNIST dataset to generate new handwritten digits.

```
import tensorflow as tf
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt
(x_train, _), _ = tf.keras.datasets.mnist.load_data()
x_train = (
    x_train.astype("float32") - 127.5
) / 127.5
x_train = np.expand_dims(
    x_train,
    axis=-1
)
batch_size = 128
buffer_size = 60000
train_dataset = tf.data.Dataset.from_tensor_slices(
    x_train
).shuffle(buffer_size).batch(batch_size)
def build_generator():
    model = tf.keras.Sequential([
        layers.Dense(
            7 * 7 * 128,
            input_dim=100
        ),
        layers.LeakyReLU(0.2),
        layers.Reshape(
            (7, 7, 128)
        ),
        layers.Conv2DTranspose(
            64,
            (5, 5),
            strides=2,
            padding="same"
        ),
        layers.LeakyReLU(0.2),
        layers.Conv2DTranspose(
            1,
            (5, 5),
            strides=2,
            padding="same",
            activation="tanh"
        )
    ])
    return model
def build_discriminator():
    model = tf.keras.Sequential([
        layers.Conv2D(
            64,
            (5, 5),
            strides=2,
            padding="same",
            input_shape=[28, 28, 1]
        ),
        layers.LeakyReLU(0.2),
        layers.Dropout(0.3),
        layers.Conv2D(
            128,
            (5, 5),
            strides=2,
            padding="same"
        ),
        layers.LeakyReLU(0.2),
```

```

        layers.Dropout(0.3),
        layers.Flatten(),
        layers.Dense(
            1,
            activation="sigmoid"
        )
    ])
    return model
generator = build_generator()
discriminator = build_discriminator()
discriminator.compile(

    loss="binary_crossentropy",
    optimizer="adam",
    metrics=["accuracy"]
)
discriminator.trainable = False
gan_input = tf.keras.Input(
    shape=(100,)
)
gan_output = discriminator(
    generator(gan_input)
)
gan = tf.keras.Model(
    gan_input,
    gan_output
)
gan.compile(
    loss="binary_crossentropy",
    optimizer="adam"
)
epochs = 5000
for step in range(epochs):
    noise = np.random.normal(
        0,
        1,
        (batch_size, 100)
    )
    fake = generator.predict(
        noise,
        verbose=0
    )
    real = x_train[
        np.random.randint(
            0,
            x_train.shape[0],
            batch_size
        )
    ]
    X = np.concatenate(
        [real, fake]
    )
    y = np.concatenate([
        np.ones((batch_size, 1)),
        np.zeros((batch_size, 1))
    ])
    d_loss, _ = discriminator.train_on_batch(
        X,
        y
    )
    noise = np.random.normal(
        0,
        1,
        (batch_size, 100)
    )

```

```
y_gen = np.ones(
    (batch_size, 1)
)
g_loss = gan.train_on_batch(
    noise,
    y_gen
)
if step % 500 == 0:
    print(
        f"Step {step} | "
        f"D loss: {d_loss:.3f} | "
        f"G loss: {g_loss:.3f}"
    )
    z = np.random.normal(
        0,
        1,
        (1, 100)

    )
    gen_img = generator.predict(
        z,
        verbose=0
    )[0, :, :, 0]
    plt.imshow(
        gen_img * 0.5 + 0.5,
        cmap="gray"
    )
    plt.axis("off")
    plt.show()
```